

Wacky Misadventures in Detecting Watermarks



nutbutter

Jan 6, 2025

ML RESEARCH

WATERMARKS

WATERMARK-DETECTION

RESEARCH

OPEN-MODEL

DEMO

+ 3



One of the areas I want to improve in [JoyCaption](#) before a 1.0 release is its accuracy in detecting watermarks. To help with that I need a secondary model that can help update my dataset. Unfortunately, there are no good watermark detection models publicly available that I'm aware of. So I embarked on a journey to build one, and I'm sharing all of the fun details and adventures of building those models. As well as the models themselves. This will be a development blog/knowledge dump of sorts. It covers everything from my data collection process, to training details, and even a little jaunt down the wacky world of hiring image annotators.

What is a Watermark

First and foremost, I needed to figure out exactly what I will consider a "watermark" for this project. The primary purpose of JoyCaption is to write descriptions, captions, and prompts for image generation models (whether for training or inference). Including the mention of watermarks in the text it writes is very important when using that text to train image generation models. This is because end-users typically don't want watermarks in their gens. If watermarks are *always* mentioned correctly during training, the model will learn to never generate one unless *explicitly* asked to.

With that in mind, what constitutes a watermark needs to be based on what users of image generation models *don't* usually want included in their gens.

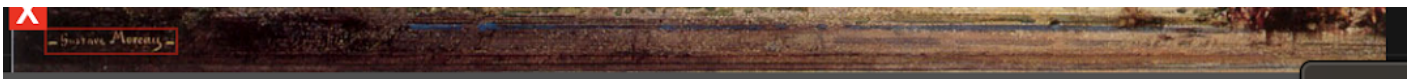


This is a good example of what traditional watermarks look like. Some kind of text or logo overlaid onto an image to either identify the source of the image, or prevent its reproduction. Note that in the case of this image I marked the entire black bar at the bottom as a watermark and not just the text. The black bar isn't part of the original image, it's part of the watermarking process, so the whole thing needs to be annotated.



This image has a typical watermark highlighted in the top right. However, in the lower right there's also an "artist signature." Users usually don't want those in their gens either.





Even signatures on traditional media should be included.



 **Q.BADSHEEPART** 

Username, social media handles, etc are also something users usually don't want in their gens.

So, overall, the definition of "watermark" should include all things like those examples. Any part of the image that isn't part of the underlying image: overlaid logos, usernames, artist names, signatures, etc.

SOTA

There may be better watermark detection models out there, but to my knowledge the best I've found is [OWLv2](#). OWLv2 is an open vocabulary object detection model from Google; think of it like YOLO but you can prompt for the objects you want detected rather than being limited to the fixed vocabulary of whatever YOLO was trained to detect. In my tests, OWLv2 with the simple prompt of "a watermark" is able to achieve an accuracy close to 90%, despite not being specifically trained on that task. Incredible!

OWLv2 was used in the training of [bigASP](#) to update all of its captions to include mentions of watermarks when they occur. This was a great proof of concept to show that if watermarks are accurately mentioned in the training dataset, diffusion models will rarely generate them unless prompted to explicitly.

For JoyCaption though, I wanted the accuracy to be a bit higher, and to ensure that the accuracy was consistent across domains: photos, traditional media, anime, vector art, etc.

First Steps

My first stab was to simply go through 1000 images from JoyCaption's dataset and tag them as either "watermark" or "no_watermark". From this curated dataset I could then finetune a classification head on top of a pretrained vision model.

Which pretrained model to train on top of? While the so400m model that JoyCaption uses is a very strong vision model, it is limited to 384x384 resolution. Watermarks can be incredibly tiny, so resolution is important here. I wanted vision models that could do something closer to 1MP of resolution, which matches the typical image generation resolution. Unfortunately, that is quite rare. All the best vision models tend to cap out around that 384x384 range. Even my own vision model, JoyTag, only goes up to 448x448.

That mostly narrows things down to OWLv2 and SAM. There are others, of course, but I looked for things I was familiar with and had good real world performance. Both are close to or at 1024x1024, and both are strong models. I performed classification head training on top of both.

SAM2 was ... very weak. It struggled to get above 60% accuracy, despite various efforts to tweak the configuration of the classification head and training setup. I think the issue here is that SAM2 is focused on segmentation. While it does well at finegrained tasks like that, the image representations that it has learned are not as useful for classifying the image as a whole. It would require significantly more training and data to build a transformer layer or two on top of its representations to shift it to the new task.

OWLv2 was a much better student, though it was a bit sensitive to the exact configuration of the classification head. OWLv2 is like typical transformer based vision models, breaking the image into a set of patches and projecting those patches into "tokens" that can then be run through your typical transformer stack. At the end is a sequence of processed tokens. In old-school CLIP models, the first or last token is extracted and projected to get the final image embedding. This is called the "class token", and the model is expected to accumulate all of its understanding of the image into it. In newer style models a pooling layer of some kind is used to pool *all* of the tokens together and then give a final embedding. There's little difference, other than pooling layers being slightly more accurate and performant. What's important to note here is that, based on the architecture and task, we would expect that CLIP-like models are using all of their tokens to understand the image as a whole. Given the global nature of attention, the tokens do not necessarily need to stay aligned with the patches they were created from. Any given final token is likely to represent some kind of global view of the entire image.

OWLv2, on the other hand, does processing on each of its tokens individually. Each token is taken and considered against the prompt to predict a bounding box and a confidence for that bounding box. So if, for example, OWLv2 had 1000 tokens, it would predict 1000 boxes, one for each token. Again the global nature of attention means that the tokens don't *necessarily* align with where they started from in the image, but because of OWL's architecture and task, it *does* mean that the tokens *must* represent discrete salient pieces of the image. If they didn't, it wouldn't be able to predict objects.

So we expect that OWL's final tokens are really a sequence of "potential objects" it has found, from which the desired objects can be detected and bounded. Some of those potential objects might be watermarks. And that's what we want the classifier head to find. It shouldn't care *where* the watermark is, just that one (or more) was detected. That means that if we used a typical classifier head that uses something like average pooling, performance might suffer. The watermark tokens would have to scream quite loudly to be heard over the average. In contrast a *max* pooling layer would allow those to be easily heard. If "watermarkness" features are detected *anywhere* they'll come through in the max pool.

And that's exactly what I saw; a huge boost in performance when I switched to max pooling. The classification head was thus:

- Linear on top of each token
- Activation function
- Max pooling
- Final linear projection for classification (watermarked or not watermarked)

With the usual sprinkling of layernorm and dropout. The theory here is that the first linear + activation extracts "watermarkness" features from each token individually. The max pool then tells us if any watermarkness features were found anywhere. The final layer gives the head an opportunity to then consider those multiple detected features together and make a final classification prediction.

With some tuning of various hyperparameters, OWLv2 with this classification head achieves an accuracy of 95%!

Now, I should mention here that a transformer based classification head might perform better here, as it could perform much more complicated transformations and accumulations on the OWLv2 tokens. However it is likely that this would require significantly more data than what I have available. Two simple linear layers can usually be trained with as little as 200 examples, but a full transformer layer would likely need something on the order of 10,000 or more to achieve better accuracy (if any).

I do apply augmentation to the training images; random flips, rotations, hue, saturation, brightness, etc. But no augmentations that clip the image in anyway, as that could clip off the watermark. In practice the augmentations aren't that helpful, since the base model is already trained to be invariant and the training of the classification head is very short so there's no much opportunity for overfitting.

Final hyperparameters:

- AdamW
- Weight decay: 0.1
- Batch size: 64
- Clip grad norm: 1
- Dropout: 0
- AMP: bfloat16
- Learning rate: 2e-4
- Cosine schedule
- 8000 training samples seen (125 batches)
- ~2000 images (50/50 split between watermarked and non-watermarked)
- 2,000 warmup samples (31 batches)

Final validation metrics:

- 100 validation images (50/50 split)
- True positives: 49
- False positives: 4
- True negatives: 46
- False negatives: 1
- Threshold: 0.5
- Accuracy: 95.00%
- Precision: 0.9245
- Recall: 0.9700
- F1: 0.9515

NOTE: I opted to train on the smaller OWLv2 B/16, rather than the larger L/14. I found the performance comparable under these conditions.

Trying to achieve a higher accuracy here would likely take significantly more data. I'm able to tag about 1000 images in day for this task, but it's exhausting work.

Bounding Boxes

Training a classification head on top of OWLv2 was very successful, but it leaves an obvious question: if OWLv2 is an object detection model that usually outputs bounding boxes ... why not train it to do *that*? Why not get bounding boxes for watermarks?

Bounding boxes around watermarks would certainly be much more useful! For training diffusion models, the bounding boxes can be used to do loss masking. For JoyCaption they can be used to help refine how it mentions the watermarks, by providing exact areas for the watermarks.

But perhaps more importantly is that bounding boxes provide *insight* into the model. With a simple classification head, we can only guess as to why the model thought that image did or did not have a watermark. With some advanced tooling we could visualize the attention maps and maybe "see what the model sees" but that kind of work is difficult. Whereas with bounding boxes, the model itself tells us what it's seeing. Additionally, I suspect that vision models can learn better representations from bounding boxes than they can from just labels. The bounding boxes tell the model exactly which pixels are of interest for a particular object.

With all of those benefits it may seem obvious to pursue training OWLv2 to output watermark bounding boxes, but that ... requires that I *have* bounding boxes to train it on. While I can label 1000 images as watermarked or not in one day of work, I can only do bounding boxes for maybe 200 in a day. It's very tedious work, even when model assisted.

The second problem is that there's no training code for OWLv2. I don't think anyone has finetuned it? It's certainly possible, but just not straightforward since it involves some rather tricky loss functions. You basically have to sort OWL's predictions first, since it will predict multiple conflicting boxes and the loss should be based off the "closest" predictions it makes. Then you need a loss function that takes into account both the accuracy of the box and the label. Again, nothing too crazy, but it's not a common and straightforward loss function, and there are some hyperparameters to tune there. OWLv2 was also trained with a somewhat complicated data pipeline. That *might* not be needed for small finetunes like this. Anyway, since this watermark detection is a *sub* project of JoyCaption, I'm not interested in pouring that kind of work into it.

YOLO, on the other hand, is stupidly easy to train thanks to the work of Ultralytics. So, I figured, why not try it? I annotated 200 images and began my journey of training YOLOv11.

I quickly discovered that YOLO is *not* a good student. My initial attempts only got mAP50 (roughly equivalent to classification accuracy) to 70%. Fiddling with hyperparameters didn't really help much, and neither did the large versions of YOLO. Performance capped out at the Medium sized model at this data scale.

Despite this, bounding boxes were too interesting to give up on. With more manual annotations (400) and more tuning I was able to get the mAP50 to 80%. Still not good enough.

And so the third leg of my journey began...

Hired Help

By this point I was tired and got curious what it would cost to have someone else do the annotation work. It wasn't particularly difficult work, just tedious. Easily something a human annotation service could handle, right? A little googling and ChatGPT later and I had an absolutely giant stack of options. Seems like this is a thriving field! Everyone and their goldfish was offering to annotate bounding boxes on images. And cheap! The going rate seemed to be \$0.03 per bounding box. My dataset averages about 1.5 boxes per image, so a nice solid dataset of 10k images would only cost \$450. I likely didn't need that many images annotated to get to a reasonable accuracy. \$100 spend? \$200 spend? Not too bad for a cool little tool.

I quickly discovered, however, that almost all of these annotation services are a "call us!" "email us!" kind of deal. No open sign up. Most barely had pricing listed! Google apparently used to have a nice service where you could just use an API to submit images and instructions and get back human annotations. Fantastic! But it doesn't seem to be offered anymore...

Labelbox seemed to be open signup and had public pricing. I was able to create an account and get my data and instructions set-up. But then when I went to start the project I got slapped with the

hard news. You can only hire in increments of a WEEK at a time. That's right, for \$350 you can hire an annotator for a week. Nothing less than that. Not billed per annotation. Ugh.

I spent an entire day going through different offerings, filling out contact forms, the works. After all of that I got only two emails back... neat. Of those two emails, neither panned out. Both went back and forth several times with mostly fill-in-the-blank sales emails from them. Much ado for such a simple project. I opted not to continue either of those chains after awhile.

One service, Supa, did have open sign up. In fact, they offered a free trial. Not bad, right? I was able to upload my dataset, instructions and ... to my absolute disbelief their workforce started annotating the images! No money exchanged, no credit card even entered. Wild.

I was initially quite hopeful for this. It was exactly what I was looking for, with the ability to just upload my images and get annotations back. I could watch the work being performed live, and even review the work and send mistakes back for rework. The overall interface was a bit janky and glitchy, but it was serviceable. The first worker to start on the dataset was doing an AMAZING job, even better than me. They found watermarks I missed, made good inferences about what is and isn't a watermark, etc. Supa had higher per bounding box pricing, but given the quality I was seeing and the free trial credits ... I was strongly considering it.

But then several other workers hopped on and quality fell quickly. The other annotators were not as good and unfortunately I saw no way to pick which people to work with. And since it was a faceless mass of workers, there was no way to really train them up. And finally, after about 200 annotations, I got booted from the service entirely due to the NSFW content in the dataset. Their rejection email was friendly and offered to refer me to a service that could better serve my needs, but I never got a reply after that... RIP.

At this point I decided to give Fiverr a try. I know, I know, fiverr is a minefield. But I was able to find five different workers with good (4.8+) and plentiful (>100) reviews. I figured I could talk to them and do some trial runs. Shouldn't be much risk.

All of them got back to me, most very quickly, which is more than can be said for all the "professional" services I ran into. And pricing was very good. With some discussion all of them quoted \$0.0125 per bounding box for my project. Nice!

With further discussion two of them fell off (no more replies) for unknown reasons. I did trial runs with the remaining three. Two used CVAT as the annotation/collaboration tool, and one used Roboflow (at my request).

I had never used CVAT or Roboflow before. CVAT seems to be the most preferred tool by these workers, but I found it to be very confusing to use personally. And for monitoring the work and

asking for revisions it was obnoxious to use. Since I was doing this as a "manager" of sorts the latter was most important to me and I did *not* like it from that perspective.

Roboflow was also very confusing to use, but had an overall more modern interface and more AI assisted tools for annotators to use. On the reviewing side it was *much* better for me. I could watch the work being done to monitor progress. Annotations could be quickly reviewed and approved or rejected to be reworked.

As an aside, Roboflow also offered their own annotation teams, as well as tools for training models on the site. I did inquire about their annotation services, and did get a reply, but it was a generic reply email that didn't actually enable me to hire any of their workers... And as for their model training, it had a nice interface but the results were abysmal. There were also various community models shared on their site, but none of the watermark trained ones worked. Seems like a nice *idea*, but poor execution.

Anyway, back to hiring Fiverr workers. After the three trial runs concluded I went through all of the results by hand. Out of the three, only one worker had good bounding boxes. The rest were very sloppy. And all three had poor overall accuracy when it came to detecting the watermarks. In fact, all of them had worse accuracy than the YOLO model I had finetuned so far...

In the end I was out \$10 and three days of working back and forth with people and services. All with nothing to show for it. I salvaged a small amount of the trial data by manually reviewing it all and fixing up the bounding boxes. Most of the data had to be thrown away. And of the data added, most of the boxes were sloppier than the ones I do. Ugh.

The Final Leg of the Journey

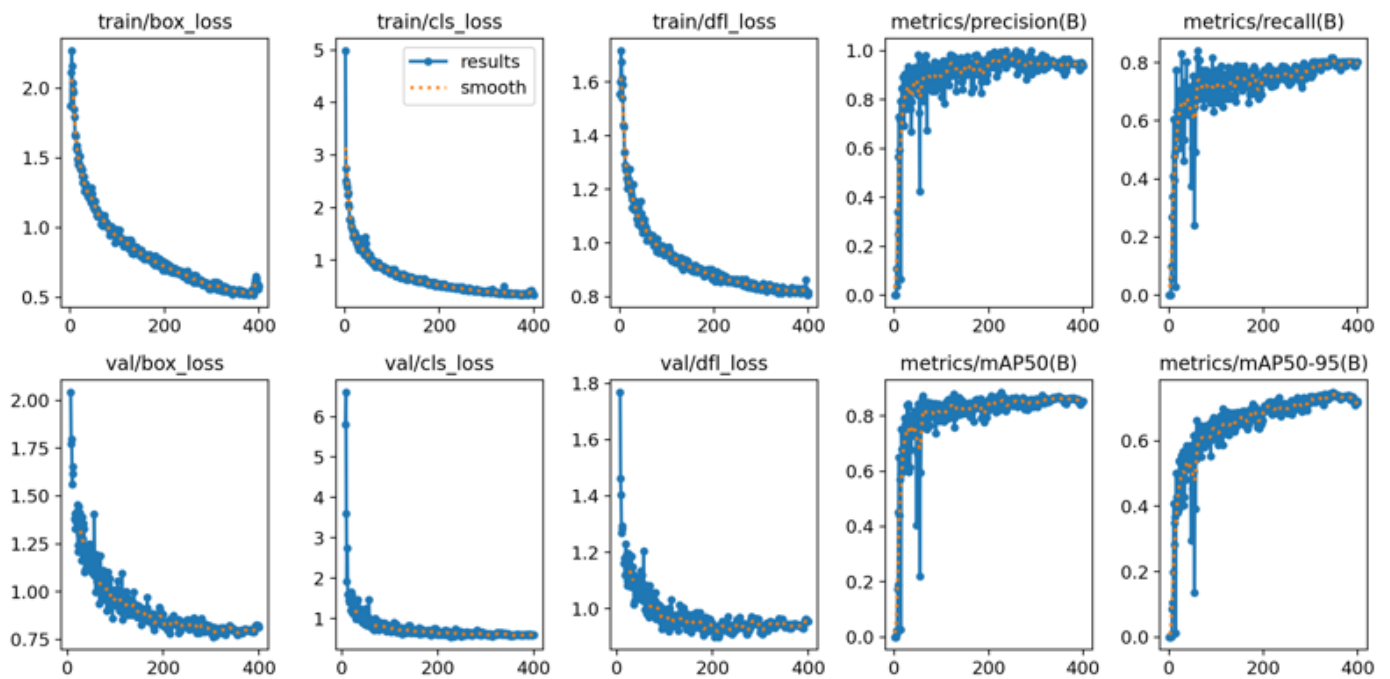
With YOLO at 80% I was at least able to use it to partially assist me in continuing to do manual annotation. A couple days of work and I'm now at just under 1000 images annotated with boxes.

The good news is that the additional data *did* help. In fact it's enough that I now see benefit from the largest YOLO model (yolo11x). At this point I have YOLO's validation accuracy up to about 90%, and mAP50 at a little below that. *Phew*

YOLO Validation Metrics

- Validation Images: 452 (50/50 balanced)
- Threshold: 0.5
- Accuracy: 89.6%
- Precision: 0.8776
- Recall: 0.9203

• F1: 0.8985



It's nowhere near my work with OWLv2, but it *does* have the benefit of giving back bounding boxes, so I don't consider the work as a whole was a waste. Probably if I could get the dataset size up further (4k? 10k?) the accuracy could approach OWLv2.

The Models

Here are the models as they are today: <https://huggingface.co/spaces/fancyfeast/joycaption-watermark-detection>

That HF space will run both, showing YOLO's box predictions as well as the classification trained OWLv2's prediction of whether or not the image as a whole has a watermark. If you go into the repo for the space (<https://huggingface.co/spaces/fancyfeast/joycaption-watermark-detection/tree/main>) the models can be downloaded and used however you wish. The space's code should show how to use both. YOLO is just standard Ultralytics YOLO11, and OWLv2 has a custom model in the code to implement the classification head.

Performance Evaluation

Inspecting OWL's mistakes on its validation data it makes the following errors:

- Doesn't see a really tricky watermark that's hidden in the image (strongly predicts no watermark).
- Strongly predicts a watermark on a very tricky negative where a website URL is shown at 50% transparency but is actually part of the image.
- Weakly predicts a watermark on a negative example; an image with speech bubbles.
- Weakly predicts a watermark on a negative example; an image of a coin. Must have false triggered on the text of the coin which kind of looks like an overlay.
- Weakly predicts a watermark on a negative example; probably false triggering on an arm patch that has logos and writing on it.

Half of those are reasonable mistakes. So with 95% accuracy and 2.5% being reasonable mistakes? A solid model overall.

Inspecting YOLO's mistakes (and having used in extensively): it's hard to give a detailed breakdown since there are 47 errors in its validation set. However, overall, it just makes a lot of really dumb errors that are hard to understand. Missing obvious watermarks, annotating random shapes on images, etc. Sometimes it nails really hard watermarks, sometimes it misses them. There doesn't seem to be any real rhyme or reason to its behavior. It also frequently false triggers on text. The OWL model will do that too (less often). Trying to discriminate text in an image from text in a watermark is a very difficult task. But YOLO in particular is quite bad at making the distinctions.

Overall, this is what I would expect from a model like YOLO. YOLO is a very small vision model. Even the largest is only 57M parameters. Compared to something like so400m which has 400M parameters. The lack of parameter count is an advantage in some contexts; it means the model is much easier to run on lower end hardware, on phones, in security cameras, etc. But it makes the

model much more difficult to finetune. Larger models learn faster, and learning faster means less data is needed to finetune them. If you check Ultralytics documents they recommend something like 10k annotations to finetune YOLO...

And for training YOLO, it had to be trained for 300 epochs to get it to 90% accuracy. That's about 360k samples seen. The OWL model only needed 8k samples seen to get to 95%.

Besides the size of the model, YOLO is also at a great disadvantage in this task being a CNN. Having worked with CNNs since the AlexNet days, I've found their behaviors to always match what I see here. Their errors are never really reasonable or explainable. Just kind of random. They don't scale. They don't have good global understanding. They learn very slowly. They don't handle pretraining well. The representations they build aren't robust. Etc, etc. In contrast, ViTs build very robust, adaptable representations. They inherently understand global context. They learn quickly (so much so that it's a problem) and can scale infinitely. They are great pretrainers. And their mistakes better align with human intuition.

Of course, the downside of transformer archs is that their training data-accuracy curves start off much lower than CNNs like YOLO. Models like YOLO can be trained from scratch on very little data. Robust CLIP models get trained on 400M+ images. JoyTag, a transformer based vision model trained from scratch, was trained successfully on 4M images but required *extensive* tuning and augmentation to get there.

All of this is to say, I doubt the YOLO based watermark detector will ever beat OWLv2 in raw accuracy. I also feel that the OWLv2 based classifier will better handle out of domain data; another thing well trained ViTs are good at and CNNs flop at. The YOLO model will really only ever be able to reliably detect watermarks in the kinds of images it has been trained with. I tried to keep the dataset as diverse as possible, but there are limits of course.

Conclusion

The model is published and public. I think I'm about done training it, since I really need to get back to the other parts of JoyCaption that need work. These models will be used to update JoyCaption's existing caption dataset to ensure they have better watermark accuracy, thus improving JoyCaption's watermark detection accuracy.

Hiring image annotators was a total flop, and honestly it explains a lot about why so many ML datasets have so many issues. "You get what you pay for" may apply here, but to be honest I can do roughly 75 annotations per hour, and those are nearly 100% pixel perfect annotations. \$5/hr is probably a reasonable non-abusive global pay, and that works out to about \$0.06 per annotation. A professional could work faster than me, driving the cost even lower. So if researchers are paying

the going rate of \$0.03 per annotation and getting the quality I saw from global workers, they're getting ripped off.

YOLO is a poor student. I have a feeling that if OWLv2 were as easy to finetune as YOLO, it would completely transform the object detection landscape. The data requirements to train it should be significantly lower, lowering the cost to create datasets and finetuned models. Its open vocabulary pre-training allows it to be used as a great starting point for helping to create datasets. And since it doesn't require as much data and trains faster, researchers can iterate with it faster. Annotate 200 images with zero-shot, train, annotate another 200 with the trained model, rinse and repeat, with each step requiring less and less effort from annotators. If inference speed is needed, a highly accurate OWLv2 finetune can be used to automatically label a very large dataset that can be used to train a high quality YOLO.

CVAT is wack. Roboflow is *okayish*.

Well, that should be about the sum of it. Maybe these watermark detectors will be useful for other people, so feel free to use them. I hope all these details about the process of building these models was interesting and perhaps helpful to others.

Be good to each other and build some cool shit.

+ 😊 👍 13

⚡ 142 📖 21 ➦

Comments

You must [sign in](#) to add a comment



qnt_iler 8 months ago



Thanks so much for another brilliant article !

+ 😊 👍 2 ❤️ 1



an303042 8 months ago



Great write up! Thank you! There certainly are ways to streamline dataset collection, and dealing with watermarks can be a big part of it 🥰

+ 😊 👍 2



septagon 8 months ago



Yeah this perfectly describes the pain I've been through in the past few months 🤯

But with a segmentation model!



nutbutter

OP

8 months ago



I can't even imagine the tedious pain of annotating for segmentation!

+ 😊 👍 3



xamy



8 months ago



Just a heads up we (friends I work with) started on segmentation masking ages ago and I already have 85k instances of reviewed and hand annotated, the quality is near perfect. Aim is to release. We should talk to avoid reinventing the wheel

+ 😊 👍 4 ❤️ 1



septagon



8 months ago



What objects are you segmenting?

+ 😊 👍 3



xamy



8 months ago



Current segmentation annotations cover: text english class and jp/cn/kr class, speech bubbles, signatures, logos and banner. Trivial to exclude or convert one class to another.

+ 😊 👍 3



septagon



8 months ago



Yeah seems like we have the same goal. Let me know when you release! Happy to lend a hand if you need help

+ 😊 👍 2

----- [Show 1 More](#) -----



LusciousFawkes

8 months ago



This is great, nice work. It should be noted that you do have a active and capable audience here, willing to help with all this hard work. I've literally done the same thing in trying to train LORAs and other detection models for watermarks, and bounding boxes aren't hard but SUPER tedious and definitely...

[Show more](#)

+ 😊 👍 3



nutbutter

OP

8 months ago



Heh, I do have people reach out and offer to help, and I do appreciate that. Stuff like contributing images to the dataset, etc. But when it comes to the grindy stuff like manually writing captions, tagging images, annotating bounding boxes, etc, nobody wants to do that. I'll go through the process of settin...

[Show more](#)

+ 😊 👍 3 😬 1



BetterPorn

8 months ago



Nice

+ 😊 👍 2

 **daniqqq** 8 months ago

This is great work and opened me some new things I did not know.

About captioning. I've always been thinking implementing some website where captioning and tagging is gamified. Everyone can register and caption it, where VLMs will work as validators and judges. When user...

[Show more](#)

+ 😊 👍 3 ❤️ 2

 **redcreamsoda** 8 months ago

Amazing work and amazing article! (and I love the "Be good to each other and build some cool shit." vibe :D)

...

[Show more](#)

+ 😊 👍 3

 **harmonicdiffusion** 8 months ago

This was very informative and greatly appreciated. Excellent write up, ideas, conclusions. I hope you post more research like this, was a great read

+ 😊 👍 2

 **[deleted]** 8 months ago

I saw on fancyfeast's page that there is work being done on style detection as well? Is there any chance we could ever see an Asp model trained on also on highly diverse anime-sourced topics, granting it illustrious/noob/pony capabilities without contaminating the photorealism that makes Asp great?

+ 😊 👍 1

 **nutbutter** OP 8 months ago

I do want to try training a model with a mixed photoreal and anime dataset at some point. Much later down the road though.

+ 😊

 **[deleted]** 8 months ago

Yeah, it would offer amazing flexibility. Currently have to use Asp as a refiner for "realism" models to use certain concepts. Anyway, looking forward to the next model whenever it appears. It's like a surprise holiday each time. Hoping for an even spicier dataset in the next model, but I'm already...

[Show more](#)

+ 😊

 **nutbutter** OP 8 months ago

Yeah I was experimenting with noise schedules to help fix some of bigASP 2's issues, but that rabbit hole was getting too deep and I decided to cut my losses and refocus on moving forward. None of the checkpoints from those experiments would really be worth publishing.



+ 🗑️ 1 [deleted] 8 months ago



The linked [paper](#) from the training [discussion](#) could be a new direction for fine-tuning.

+ 😊



firemanbrakeneck 7 months ago



Naive question, couldn't the owl2 classifier be used for (roughly) bounding the watermarks by zooming in regions where they are detected (as in a quadtree)? Might help that watermarks tend to have mostly fixed locations (corners or center). Though I'm not really seeing that 90%.

+ 😊



themex 6 months ago



Really informative article! Found some clues on why some of my own experiments weren't going anywhere. Thank you so much.

+ 😊



reaper557 6 months ago



Great article! Thank you very much for sharing these models and hard-won information. 31(̂ ̂ ̂ - ̂) ✨

+ 😊



Gebbsfrom404 6 months ago



Have you tried label-studio? Interface is okay but you can plug-in existing yolo model to annotate dataset interactively, and with it you can approve it's prediction if it's good. You can go very fast if you already have okay model. ctrl+enter, eyeball image for a few seconds, ctrl+enter. I'm not affiliated it's just good....

[Show more](#)

+ 😊



nutbutter OP 6 months ago



I'll take a look at it, thank you :)

+ 😊



[deleted] 5 months ago



Looks like you are doing a lot of background work to maybe get ready to train another asp?

+ 😊